

FORMULARIO DE SUPERVIVENCIA: CC1002 – CONTROL 1

Felipe Colli Olea – FCFM Universidad de Chile

1. LA RECETA DE DISEÑO (PUNTAJE SEGURO)

Obligatorio en CADA función para no perder hasta 1.5 pts:

```
# nombreFuncion: tipo1 tipo2 -> tipoRetorno
# Proposito: Breve descripcion de que calcula
# Ejemplo: nombreFuncion(arg1, arg2) devuelve resultado
def nombreFuncion(param1, param2):
    # Cuerpo de la funcion
    return resultado
```

```
# Tests (Al menos 1 o 2 por funcion):
assert nombreFuncion(val1, val2) == esperado
assert not funcionBooleana(casoFalso)
assert cerca(funcionReal(val), esperado, 0.0001)
```

Tipos para contratos: int, float, num (ambos), str, bool, None (procedimientos).

2. MANIPULACIÓN DE DÍGITOS (MACHETE CLAVE)

Dado un número entero positivo N :

- Último dígito (derecha): $n \% 10$
- Quitar último dígito: $n // 10$
- Último par de dígitos (base 100): $n \% 100$
- Quitar último par de dígitos: $n // 100$
- Dígito en posición k (desde der=0): $(n // (10^{**k})) \% 10$
- Primeros 2 dígitos (izq): $n // (10^{**(\text{digitos}(n) - 2)})$
- Quitar primeros 2 dígitos (izq): $n \% (10^{**(\text{digitos}(n) - 2)})$
- Quitar ambos extremos (2 dígitos c/u):
 $\text{nuevo} = (n \% (10^{**(\text{largo} - 2)})) // 100$

3. PLANTILLAS DE RECURSIÓN CLÁSICAS

Contar Dígitos / Largo

```
# digitos: int -> int
def digitos(n):
    if n < 10:
        return 1
    return 1 + digitos(n // 10)
```

Suma de Dígitos

```
# sumaDigitos: int -> int
def sumaDigitos(n):
    if n < 10:
        return n
    return (n % 10) + sumaDigitos(n // 10)
```

Invertir un Número

```
# invertir: int -> int
def invertir(n):
    if n < 10:
        return n
    k = digitos(n // 10)
    return (n % 10) * (10 ** k) + invertir(n // 10)
```

Mayor Dígito de un Número

```
# maxDigito: int -> int
def maxDigito(n):
    if n < 10:
        return n
    resto = maxDigito(n // 10)
    u = n % 10
    return u if u > resto else resto
```

Procesar Secuencia por Bloques de 2 Dígitos

```
# sumarPares: int -> int
def sumarPares(sequencia):
    if sequencia == 0:
        return 0
    return (sequencia % 100) + sumarPares(sequencia // 100)
```

Procedimiento Recursivo (Imprimir sin retornar)

```
# mostrarPares: int -> None
def mostrarPares(n):
    if n > 0:
        print("Valor:", n % 100)
        mostrarPares(n // 100)
```

Comprobar Simetría de Extremos (Estilo Sumeru)

```
# comprobar: int -> bool
def comprobar(N):
    largo = digitos(N)
    if largo == 4:
        return True
    # Extraer extremos
    der = N % 100
    izq = N // (10 ** (largo - 2))
    # Reducir número eliminando ambos extremos
    nuevoN = (N % (10 ** (largo - 2))) // 100
    if (izq + der) != sumarExtremos(nuevoN):
        return False
    return comprobar(nuevoN)
```

Función Wrapper con Acumulador (Recurción de Cola)

```
# helper: int int -> int
def helper(n, acc):
    if n == 0:
        return acc
    return helper(n // 10, acc + (n % 10))
```

funcionPrincipal: int -> int

```
def funcionPrincipal(n):
    return helper(n, 0)
```

4. COMPARACIÓN DE FLOTANTES (CERCA)

¡Nunca usar == para números reales (IEEE 754)!

```
# cerca: num num num -> bool
def cerca(x, y, eps=0.0001):
    return abs(x - y) < eps
```

```
assert cerca(0.1 + 0.2, 0.3)
```

5. MÓDULOS Y ENTRADAS/SALIDAS

Módulo Math y Random

```
import math, random
math.sqrt(x) # Raiz cuadrada
```

```
math.pi          # Constante pi
round(num, n)     # Redondea a n decimales
random.randint(a, b) # Entero aleatorio en [a, b]
random.random()   # Float aleatorio en [0.0, 1.0)
```

Interacción Básica

```
x = int(input("Ingrese entero: "))
y = float(input("Ingrese real: "))
# Concatenacion limpia sin errores de tipo:
print("Resultado: " + str(x) + " y " + str(y))
```

6. TRUCOS Y ERRORES TÍPICOS A EVITAR

- **NOTA DEL ENUNCIADO:** En casi todos los controles prohíben explícitamente usar `str()` para procesar números en las preguntas de recursión. Usa solo `% 10`, `// 10` y potencias de 10.
- **Condicionales directos:** En vez de `if x% 2 == 0:`
`return True` else: `return False`, escribe directo:
`return (x% 2 == 0)`.
- **Importar módulos de la pregunta:** Si el enunciado dice que existe un módulo `domino` o `musical`, no olvides escribir al inicio:
`from domino import *`
- **Pre-condición con `assert`:** Si te piden que $x > 0$, mete un `assert x > 0` dentro del cuerpo antes de la lógica.